

Database Systems laboratory Work

Part 1: Key Identification Exercises

Relation A: Employee

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

Task 1. List 6 different superkey:

Answer:

1. {EmpID}
2. {SSN}
3. {Email}
4. {SSN, Phone}
5. {EmpID, Name}
6. {Email, Department, Salary}

Task 2. Identify all candidate keys

Answer:

{EmpID}, {SSN}, {Email}

Task 3. Which candidate key should be used as primary key and why?

Answer:

EmpID should be used as the primary key because it is unique, simple, stable, and does not contain personal information.

Task 4. Can two employees have the same phone number?

Answer:

Yes, two employees can have the same phone number. The sample data shows different phone numbers, but there is no business rule saying that Phone must be unique. For example, two employees may share the same office phone.

Relation B: Course Registration Analysis

- **1. Minimum attributes for Primary Key:**
 - {StudentID, CourseCode, Section, Semester, Year}
- **2. Why each attribute is necessary:**
 - StudentID: Identifies the student.
 - CourseCode: Identifies the subject course.
 - Semester & Year: Allows a student to retake a course section in a different term/year.
- **3. Additional Candidate Keys:**

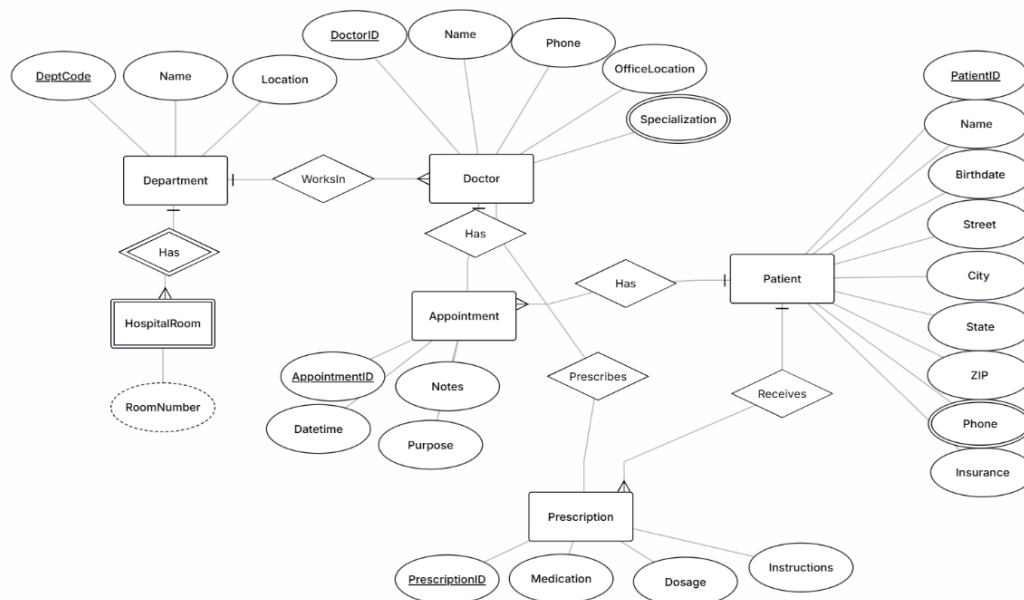
- None. Any proper subset of {StudentID, CourseCode, Section, Semester, Year} would violate the business rules (e.g., omitting Semester prevents re-taking courses, omitting Section allows double-enrollment in different sections of the same semester).

Task 1.2: Foreign Key Design

- Student.AdvisorID - Professor.ProfID
- Course.DepartmentCode - Department.DeptCode
- Department.ChairID - Professor.ProfID
- Enrollment.StudentID - Student.StudentID
- Enrollment.CourseID - Course.CourseID

Part 2: ER Diagram Construction

Task 2.1: Hospital Management System



1. Complete ER Diagram

The ER diagram includes **Patient**, **Doctor**, **Department**, **Appointment**, and **Patient Phone**.

2. Weak Entity

Patient Phone is a weak entity because it depends on **Patient**.

3. Many-to-Many Relationship

Patient–Doctor is an M:N relationship. It has attributes: **date**, **time**, **reason**, and **status**, represented by **Appointment**.

Task 2.2: E-commerce Platform

1. Complete ER Diagram

The ER diagram includes the following entities: **Customer**, **Order**, **Product**, **Category**, **Vendor**, **Order Item**, **Review**, **Inventory**, and **Address**, with their corresponding attributes and relationships.

2. Weak Entity

Order Item is a weak entity because it depends on an **Order** for its existence and identification. It cannot exist independently without a specific order.

3. Many-to-Many Relationship

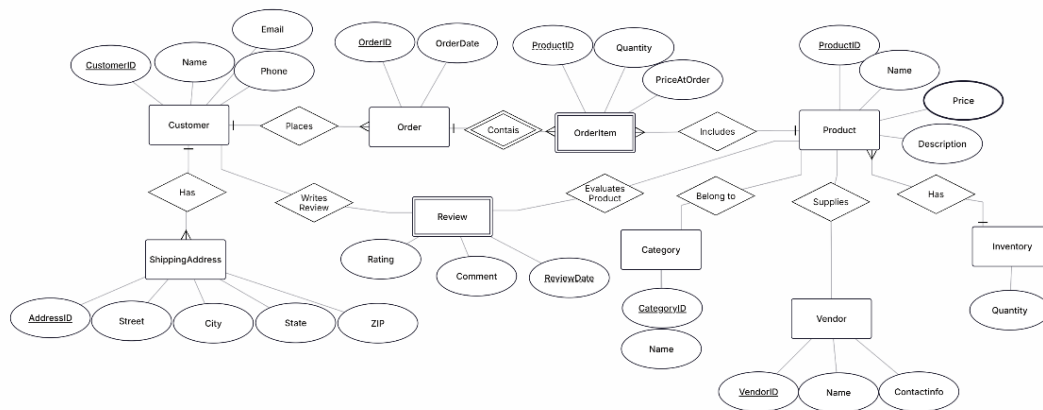
Order and **Product** have a many-to-many relationship because one order can contain many products, and one product can be included in many orders.

The relationship has attributes:

Quantity

price_at_order

This M:N relationship is represented by the **Order Item** entity.



Task 4.1: Denormalized Table Analysis

1. Functional Dependencies

StudentID → **StudentName**, **StudentMajor**

ProjectID → **ProjectTitle**, **ProjectType**, **SupervisorID**, **StartDate**, **EndDate**

SupervisorID → **SupervisorName**, **SupervisorDept**

(**StudentID**, **ProjectID**) → **Role**, **HoursWorked**

2. Problems

Redundancy:

Student and project information is repeated in multiple rows. For example, the project title and supervisor information are repeated for every student working on the same project.

Update anomaly:

If a supervisor changes department, we need to update the department in many rows. If one row is not updated, inconsistent data appears.

Insert anomaly:

A new project cannot be easily added if no student is assigned to it yet.

Delete anomaly:

If the last student working on a project is deleted, information about the project and its supervisor may also be lost.

3. 1NF

The table is already in **1NF** because all attributes contain atomic values and there are no repeating groups or multivalued attributes.

4. 2NF

The primary key is:

(StudentID, ProjectID)

There are partial dependencies:

StudentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate

Therefore, the table is decomposed into:

Student(StudentID PK, StudentName, StudentMajor)

Project(ProjectID PK, ProjectTitle, ProjectType,
SupervisorID, StartDate, EndDate)

StudentProject(StudentID PK/FK, ProjectID PK/FK,
Role, HoursWorked)

5. 3NF

There is a transitive dependency:

ProjectID $\rightarrow$ SupervisorID

SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

Therefore:

ProjectID $\rightarrow$ SupervisorName, SupervisorDept

The final 3NF decomposition is:

```

Student (
    StudentID PK,
    StudentName,
    StudentMajor
)

Supervisor (
    SupervisorID PK,
    SupervisorName,
    SupervisorDept
)

Project (
    ProjectID PK,
    ProjectTitle,
    ProjectType,
    SupervisorID FK,
    StartDate,
    EndDate
)

StudentProject (
    StudentID PK/FK,
    ProjectID PK/FK,
    Role,
    HoursWorked
)

```

Task 4.2: Advanced Normalization

1. Primary Key

A student can be enrolled in multiple course sections. Therefore, **StudentID alone is not a key**.

Assuming `CourseID + TimeSlot` identifies a course section, the primary key is:

```
(StudentID, CourseID, TimeSlot)
```

2. Functional Dependencies

```
StudentID → StudentMajor
```

```
CourseID → CourseName
```

```
InstructorID → InstructorName
```

```
Room → Building
```

```
(CourseID, TimeSlot) → InstructorID, Room
```

3. BCNF

The table is **not in BCNF** because some determinants are not superkeys.

For example:

```
StudentID → StudentMajor
```

StudentID is not a superkey because one student can take multiple courses.

Also:

CourseID $\rightarrow$ CourseName

CourseID is not a superkey because many students can take the same course.

Therefore, the relation violates **BCNF**.

4. BCNF Decomposition

The table can be decomposed into:

```
Student(  
    StudentID PK,  
    StudentMajor  
)  
  
Course(  
    CourseID PK,  
    CourseName  
)  
  
Instructor(  
    InstructorID PK,  
    InstructorName  
)  
  
Room(  
    Room PK,  
    Building  
)  
  
CourseSection(  
    CourseID PK/FK,  
    TimeSlot PK,  
    InstructorID FK,  
    Room FK  
)  
  
Enrollment(  
    StudentID PK/FK,  
    CourseID PK/FK,  
    TimeSlot PK/FK  
)
```

5. Information Loss

The decomposition is **lossless** if the tables are joined using their primary and foreign keys. The original information can be reconstructed using `JOIN` operations between Student, Enrollment, CourseSection, Course, Instructor and Room.

Part 5: Design Challenge

Task 5.1: Real-World Application

